



موږڊي MY SUPPLIER
Build for Less

Complete Developer Guide

My Supplier (موږڊي) · Build for Less

Building-materials price comparison, RFQ & bidding platform — website, dashboards, API, mobile

Repository: <https://github.com/invetechs1/My-supplier-> (branch `main`)

Version 1.2 · 17 September 2026 · Owner: Bassir Technology

Includes: website go-live gap analysis · system guide · deployment runbook · integrations · API · remaining backlog

Contents

1. Part 1 — Executive summary (Arabic / English)
2. Part 2 — What is missing for the website to go live (gap analysis, UAT, sign-off)
3. Part 3 — System overview & features (with screenshots)
4. Part 4 — Architecture & data model
5. Part 5 — Deployment runbook
6. Part 6 — Integrations & every environment variable
7. Part 7 — API reference
8. Part 8 — Remaining backlog after launch (P0 / P1 / P2)
9. Part 9 — Business model & roadmap
10. Part 10 — Brand
11. Appendix — Demo accounts, commands, file layout

Part 1 — Executive summary

الوضع الحالي: النظام مكتمل التطوير ومختبر ومنشور على GitHub بصورة Docker جاهزة. الموقع بكل لوحاته (المشتري، المورد، الإدارة) يعمل من طرف لطرف، والدفع الآمن ورموز التحقق والإشعارات مبنية وتعمل في وضع التطوير.

ما ينقص الموقع ليكون جاهزاً للإطلاق (التفاصيل في الجزء الثاني): خادم ونطاق، فتح حسابات المزودين الحقيقيين (Moyasar للدفع، Unifonic للرسائل، بريد SMTP) وضبط مفاتيحها، نصوص قانونية معتمدة (الشروط والخصوصية)، بيانات الشركة (الرقم الضريبي، الآيبان، نسبة الرسوم)، تحسين أمني بسيط، مراقبة ونسخ احتياطي، محتوى الكتالوج والموردين الأوائل، ثم اختبار قبول على الإنتاج. جهد المطور التقديري: 8–12 يوم عمل، معظمه انتظار موافقات الجهات الخارجية.

تطبيق الجوال مبني ومُراجع برمجياً لكنه لم يُشغَل على أجهزة حقيقية بعد؛ وهو مسار مستقل لا يمنع إطلاق الموقع.

Status: the platform is fully developed, tested (17 automated end-to-end scenarios), and published on GitHub with a production Docker image built by CI. The website with its three dashboards works end to end; escrow payments, OTP and multi-channel notifications are built and run in development mode with built-in stand-ins.

To go live the website needs: infrastructure (server, domain), real provider accounts and keys (Moyasar, Unifonic, SMTP), legal texts, company identity settings, a short security and monitoring pass, catalog content and first suppliers, then a UAT run. Developer effort: about 8–12 working days, most of it waiting on third-party approvals. Part 2 lists every item with owner and estimate.

Part 2 — What is missing for the website to go live

This part answers one question: **what exactly is still missing before the website can go live?**

Everything below is ordered; "Owner" items need the company (accounts, legal, money), "Dev" items need the developer. Estimates assume one experienced full-stack developer. Total developer effort to launch the website: **about 8–12 working days**, mostly waiting on third-party approvals.

A. Status today

Component	State	Evidence
Backend API (95+ endpoints)	Complete	17 end-to-end tests green on every push (GitHub Actions)
Website + buyer / supplier / admin dashboards	Complete	Every flow rendered and driven in Chromium: search, compare, RFQ, bids, award, pay, OTP, BOQ import, admin
Docker image + HTTPS compose + CI	Complete	Image builds and publishes to GHCR on every push to <code>main</code>
Payments, OTP, notifications, storage	Built, live providers not yet exercised	Verified with mock gateway / console providers only
Legal pages, robots.txt, sitemap, 404	Placeholders in place	Routes exist; legal wording must be supplied
Mobile app (iOS/Android)	Not required for the website launch	Typechecked, never run on a device — separate track

B. What is missing — ordered checklist for the website launch

#	Item	Who	What to do	Est.
1	Domain, server, DNS	Owner + Dev	Buy the domain (e.g. <code>mysupplier.sa</code> via SaudiNIC or a <code>.com</code>), a Linux VM (2 vCPU / 4 GB, KSA or Bahrain region), point A records for the root and <code>www</code> . Install Docker. Follow <code>DEPLOYMENT.md</code> §1. HTTPS is automatic.	0.5 d
2	Production environment file	Dev	Copy <code>.env.production.example</code> → <code>.env</code> ; generate <code>JWT_SECRET</code> and <code>POSTGRES_PASSWORD</code> ; set <code>DOMAIN</code> , <code>PUBLIC_BASE_URL</code> , <code>WEB_BASE_URL</code> , <code>CORS_ORIGINS</code> ; <code>SEED_DEMO_DATA=0</code> ; <code>OTP_REQUIRED=1</code> ; <code>OTP_DEBUG_RETURN_CODE=0</code> ; <code>ADMIN_PASSWORD</code> then change it after first login.	0.5 d
3	Payment gateway (Moyasar)	Owner opens account · Dev configures	Merchant onboarding needs CR, bank account, website URL. Then: live keys → <code>.env</code> ; webhook URL <code>https://DOMAIN/api/v1/payments/webhook/moyasar</code> with a secret; enable mada, Visa/MC, STC Pay; for Apple Pay upload Moyasar's domain-association file to <code>web/public/.well-known/</code> and rebuild. Test : one real order of ~SAR 5, confirm it shows paid, invoice appears, then refund from Admin → Finance.	1 d + approval time (3–10 days)

4	SMS OTP (Unifonic)	Owner opens account · Dev configures	Unifonic account + sender-ID registration with CST (needs CR; 2–7 days). Set <code>SMS_PROVIDER=unifonic</code> , <code>UNIFONIC_APP_SID</code> , <code>UNIFONIC_SENDER_ID</code> . Test: register with a real number and receive the code. Alternative for a fast pilot: Twilio.	0.5 d + approval
5	Email (SMTP)	Owner provides mailbox · Dev configures	Google Workspace / Zoho / Amazon SES mailbox no - <code>reply@DOMAIN</code> ; add SPF, DKIM, DMARC DNS records; set <code>EMAIL_PROVIDER=smtp</code> + SMTP vars. Test: registration email lands in inbox, not spam.	0.5 d
6	File storage bucket	Dev	Create an S3-compatible bucket (AWS me-south-1, Oracle Jeddah, or Wasabi); set <code>STORAGE_BACKEND=s3</code> + keys. Optional but recommended over local disk so uploads survive container rebuilds (local disk works if the <code>apidata</code> volume is kept).	0.5 d
7	Private document URLs	Dev	Supplier compliance documents are currently reachable by anyone who has the random URL. Add <code>save_private</code> + <code>signed_url</code> in <code>backend/app/services/storage.py</code> and serve documents through an authenticated endpoint (see Part E, P0).	1 d
8	Legal content	Owner (lawyer) · Dev pastes	Terms of service, privacy policy (PDPL), supplier agreement (fee %, payout timing, disputes). Pages already exist at <code>/terms</code> , <code>/privacy</code> , <code>/about</code> , <code>/contact</code> ; text lives in <code>web/src/pages/StaticPages.tsx</code> . Add a consent checkbox on registration once the text is final.	0.5 d
9	Company identity in the system	Owner · Dev	<code>PLATFORM_VAT_NUMBER</code> , legal name, <code>PLATFORM_BANK_INSTRUCTIONS</code> (real IBAN for bank transfers), platform fee % decision (<code>PLATFORM_FEE_PCT</code> , default 2.5), contact email/phone on the contact page, final Arabic mission copy in <code>web/src/i18n.tsx</code> .	0.5 d
10	Catalog and data	Owner + Dev	Start with <code>SEED_DEMO_DATA=0</code> (catalog + admin only). Review the 15 categories / 58 products in Admin, add real products (CSV import or Admin), invite the first 20–50 suppliers and verify their documents, add 2–3 external price sources so comparisons show data on day one.	2–3 d (ongoing)
11	Security hardening	Dev	Add <code>Content-Security-Policy</code> in <code>deploy/Caddyfile</code> ; confirm CORS is the domain only; run OWASP ZAP baseline scan against staging; rotate all keys used during testing; enable PostgreSQL backups (item 13).	1 d
12	Monitoring	Dev	Sentry project → <code>SENTRY_DSN</code> ; uptime monitor (Better Stack / UptimeRobot) on <code>/api/v1/health</code> ; alert email.	0.5 d
13	Backups tested	Dev	Cron <code>deploy/backup.sh</code> nightly; copy backups off-server (S3); perform one restore on a scratch database and document the time it took.	0.5 d
14	Analytics & SEO	Dev	GA4 (or Plausible) snippet in <code>web/index.html</code> ; submit <code>/sitemap.xml</code> to Google Search Console; verify Open Graph preview. Product pages are client-rendered — acceptable for launch; pre-rendering is a P1 item.	0.5 d

15	User acceptance test	Owner + Dev	Run the UAT script in section C on production with real accounts before announcing.	1 d
----	-----------------------------	-------------	---	-----

C. UAT script (run on production before launch)

1. Open `https://DOMAIN` : brand, slogan and HTTPS padlock visible; Arabic and English toggle works; mobile width layout OK (test on a phone browser).
2. Register a buyer with a real phone → OTP received → logged in → notification preferences visible in Settings.
3. Register a supplier → upload CR document → admin approves in Admin → Trust → supplier verified in Admin → Suppliers.
4. Supplier adds 3 prices in the price list (one via CSV import); prices appear on product pages with the "verified" badge.
5. Buyer searches, compares two products, sets a price alert, adds items to the quote list, imports a BOQ Excel, sends the RFQ.
6. Supplier receives the notification (in-app + SMS/email), bids; second supplier bids; buyer sees ranked comparison, exports Excel, awards.
7. Buyer pays the order with mada (real small amount) → order shows paid → tax invoice opens with QR → supplier sees the paid order and delivers → payout appears pending → admin marks payout paid → buyer rates the supplier.
8. Refund the test payment from Admin → Finance; open and resolve a test dispute.
9. Admin: dashboard KPIs, delivery log shows sent messages, jobs ran, price sources fetched.
10. Password reset by email OTP; login by phone OTP; logout; 404 page; terms/privacy links in footer.

D. Go-live sign-off

Check	Owner	Done
Items 1–15 above completed	Dev lead	☐
UAT script passed on production with real accounts	Owner	☐
Demo accounts absent; admin password changed; secrets rotated	Dev	☐
Backup restored once successfully	Dev	☐
Legal pages final; VAT number on invoices verified	Owner	☐
Support channel (email/phone/WhatsApp) staffed	Owner	☐

Part 3 — System overview & features

موردي — My Supplier · Build for Less

رسالتنا: أن يحصل كل من يبني في المملكة — مقاولاً كان أو مالك مشروع — على أفضل سعر لكل مادة بناء، بشفافية كاملة وبضغطة واحدة. كل ريال توقّره في المواد هو ريال يبني به أكثر. **Our mission:** everyone who builds in the Kingdom gets the best price on every material, transparently and in one click. *Build for Less.*

منصة أسعار مواد البناء وطلبات التسعير في المملكة العربية السعودية. Building-materials price comparison, RFQ (request-for-quotation) and supplier-bidding platform — web, dashboard, and iOS/Android app.

Layer	Stack	Folder
API + database	Python 3.12 · FastAPI · SQLAlchemy 2 · SQLite (dev) / PostgreSQL (prod) · JWT	backend/
Website + buyer/supplier/admin dashboards	React 18 · TypeScript · Vite · Arabic RTL + English	web/
Mobile app (iOS + Android)	Expo SDK 53 · React Native · React Navigation	mobile/
Docs	Architecture, API, business model, roadmap	docs/

What it does

For anyone in construction (buyers) - Search any construction item and see **every supplier's price side by side** (registered suppliers + external reference sources), with best / average / median price, 90-day price history and 30-day change. - Filter by city, brand, category; compare products; **price-drop alerts** (watchlist). - **Buy directly** from a registered supplier's offer, or collect items into a **quote list** → **RFQ** with quantities. - RFQ reaches matching registered suppliers instantly (category + city matching, or invite-only), suppliers bid, buyer sees **ranked bids with market reference**, awards with one click → order is created and tracked to delivery, then rated.

For suppliers - Register, get verified by the platform, manage a **live price list** (per city, min qty, stock, VAT handling) or **bulk import CSV/JSON**. - Receive **open RFQs** in their specialties, submit / update / withdraw bids, see win-rate, orders, pipeline and revenue on their dashboard. - Confirm → deliver orders; collect ratings.

Payments, OTP, notifications (v1.1) - Escrow payments: pay an order by mada / card / Apple Pay / STC Pay (Moyasar hosted page) or bank transfer; funds are held by the platform and released to the supplier after delivery, minus the platform fee. Payouts, refunds, ZATCA-style tax invoices with QR, platform-fee invoices, admin finance dashboard. - **OTP:** SMS/WhatsApp/email one-time codes for registration, passwordless phone login, password reset and phone verification (Unifonic / Twilio / SMTP; console mode for dev). - **Notifications:** every event goes to in-app + email + SMS + WhatsApp + Expo push according to the user's preferences, through an outbox with retries and an admin delivery log. - **Background jobs:** RFQ expiry, price alerts, daily external feed fetching, payment expiry. Rate limiting, Alembic migrations. - **v1.2:** logo/product-image/document uploads (local or S3), supplier compliance document review, Excel/CSV bill-of-quantities import into an RFQ with product matching, bid comparison export to Excel, order disputes with admin resolution and refund, Sentry hook. - Details and env variables: [docs/INTEGRATIONS.md](#).

For the platform (admin dashboard) - KPIs: users, suppliers pending verification, products, offers, RFQs, bids, orders, GMV, estimated take-rate revenue, 30-day series. - Supplier verification & plans (free / pro / enterprise), user management, category management. - **External price sources:** CSV/JSON feeds by URL or manual upload

→ appear as reference prices in comparisons ("prices from all other suppliers/resources"). - Data-quality view: stale offers, outlier prices; audit log.

Run locally

1. API

```
cd backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

- API: `http://localhost:8000/api/v1` · interactive docs: `http://localhost:8000/docs`
- First start creates the SQLite DB in `backend/data/`, the category tree, ~58 products, and (when `SEED_DEMO_DATA=1`, the default) 7 demo suppliers with prices and 90 days of history.

Demo accounts (password `Demo@2026` unless noted)

Role	Email
Admin	<code>admin@mysupplier.sa</code> / <code>Admin@2026</code>
Buyer	<code>buyer@demo.sa</code>
Suppliers	<code>supplier1@demo.sa</code> ... <code>supplier7@demo.sa</code>

2. Website + dashboards

```
cd web
npm install
npm run dev      # http://localhost:5173 (proxies /api to :8000)
npm run build    # → web/dist – served automatically by the API at / when present
```

3. Mobile app

```
cd mobile
npm install --legacy-peer-deps
npx expo start    # scan the QR with Expo Go (iOS/Android); in dev it targets http://<your-host>:8000
```

Store builds: `eas build -p ios|android --profile production` (set `apiBase` in `app.json` / `EXPO_PUBLIC_API_BASE`).

4. Tests

```
cd backend && python -m pytest -q      # 17 end-to-end scenarios (auth, catalog, import,
RFQ→bid→award→order→review, admin, payments, OTP, notifications, jobs)
cd web && npm run typecheck
cd mobile && npm run typecheck
```

Deploy

Developer handover (what is built, what remains, how to go live): [docs/DEVELOPER_HANOVER.pdf](#)

Production, step by step: [DEPLOYMENT.md](#) (server + HTTPS, Moyasar, Unifonic, SMTP, app stores, go-live checklist).

Quick version: `cp .env.production.example .env`, fill it, then `docker compose -f docker-compose.prod.yml up -d --build` — Caddy gives you HTTPS, the API image builds the web app and serves it, migrations run on start. Dev: `docker compose up --build`. A GitHub Action publishes the image to GHCR on every push to `main`.

Repository layout

backend/app/	
main.py	FastAPI app, routers, SPA hosting
models.py	Users, suppliers, categories, products, offers, price history, sources, RFQs, RFQ items, bids, orders, reviews, notifications, alerts, audit
routers/	auth · catalog · market · suppliers · rfq · orders · notifications ·
admin	
services/	pricing · ingestion (CSV/JSON feeds) · matching · notify · channels
(email/SMS/WhatsApp/push outbox)	
	payments (mock/Moyasar, escrow, payouts, invoices) · otp · jobs
(scheduler)	
backend/migrations/	Alembic migrations
seed.py	Saudi building-materials catalog + demo data
backend/tests/	pytest end-to-end suite
web/src/	pages/ (public, buyer/, supplier/, admin/), components/, api.ts,
i18n.tsx, auth.tsx	
mobile/src/	screens/, store.tsx, api.ts, i18n.ts, theme.ts
docs/	ARCHITECTURE.md · API.md · BUSINESS_MODEL.md · ROADMAP.md

تسجيل الدخول

EN

قائمة التسعير

مؤشر السوق

الموردون

الأسعار

موردي

BUILD FOR LESS

Build for Less

ابن بأقل تكلفة — أسعار مواد البناء من كل الموردين في مكان واحد

قارن الأسعار، اطلب عروض تسعير من الموردين المسجلين، واشتر بأفضل سعر لموقع مشروعك

ابحث عن أسمنت، حديد، بلوك، سيراميك...

كل المدن

ابحث

5

230

7

58

مدينة

عرض سعر حي

مورد مسجل

منتج مُسعر

4.7%

وقر عملاؤنا مقارنة بمتوسط السوق

رسالتنا

أن يحصل كل من يبنى في المملكة — مقاولاً كان أو مالك مشروع — على أفضل سعر لكل مادة بناء، بشفافية كاملة وبضغطة واحدة. كل ريال توفّره في المواد هو ريال يبنى به أكثر.

Build for Less

الفئات

الأسمنت

منتج 6

الحديد

منتج 8

البلوك والطوب

منتج 5

الرمل واليخس

منتج 4

الخرسانة الجاهزة

منتج 3

البلاط والرخام

منتج 6

الدهانات

منتج 4

السباكة

منتج 6

الكهرباء

منتج 6

الأخشاب

منتج 2

العزل

منتج 2

الجبس والأسقف

منتج 2

الأبواب والنوافذ

منتج 2

العدد والأدوات

منتج 2

أخرى

منتج 0

Home page: slogan, search, live stats, mission strip, categories

تسجيل الدخول

EN

قائمة التسعير

مؤدّي

الأسعار

المؤرّدون

مؤشر السوق

الأسعار / حديد تسليح

حديد تسليح قطر 12 مم

حديد · RB-12 · الوحدة: ton

diameter_mm: 12

1.0%

تغيّر 30 يوماً

4

عروض · 4 المؤرّدون

2,758.28

أعلى سعر

2,701.09

متوسط السعر

2,650.56

أفضل سعر (ر.س، قبل الضريبة)

#	المؤرّد	المدينة	السعر (قبل الضريبة)	شامل الضريبة	أقل كمية	التوفر	آخر تحديث
1	قائمة أسعار مورد خارجي (جدة) مصدر خارجي	جدة	ر.س / 2,650.56 ton	ر.س 3,048.14	1	متوفر	6 سبتمبر 2026
2	مؤسسة البناء الحديث لمواد البناء موثّق 4.6	الرياض	ر.س / 2,670.23 ton	ر.س 3,070.76	10	متوفر	2 سبتمبر 2026
3	مؤشر أسعار السوق — مرجع مصدر خارجي	الرياض	ر.س / 2,725.27 ton	ر.س 3,134.06	1	متوفر	5 سبتمبر 2026
4	شركة الخليج للحديد والأسمنت موثّق 4.3	الدمام	ر.س / 2,758.28 ton	ر.س 3,172.02	1	متوفر	9 سبتمبر 2026

منتجات مشابهة

2.3%

حديد تسليح

حديد تسليح قطر 8 مم

2,572.71

ر.س

متوسط السعر: ر.س 2,801.10 · 4 عروض · 3 المؤرّدون

+ أضف لطلب التسعير

حركة السعر — آخر 90 يوماً

Product page: every supplier's price side by side, VAT-normalised, with history

تسجيل الدخول

EN

قائمة التسعير

مؤدّي

الأسعار

المؤرّدون

مؤشر السوق

لوحة التحكم

طلب تسعير جديد

المدينة *

الرياض

المشروع

عنوان المورد

mm/dd/yyyy

تاريخ التوريد المطلوب

mm/dd/yyyy

الكمية

25

الوصف

حديد تسليح قطر 12 مم

الكمية

800

الوصف

أسمنت برتغالي عادي 50 كغم

الكمية

25

الوصف

حديد تسليح قطر 12 مم

الكمية

800

الوصف

أسمنت برتغالي عادي 50 كغم

RFQ form with bill-of-quantities import and product matching

تسجيل الدخول

EN

قائمة التسعير

مؤدّي

الأسعار

المؤرّدون

مؤشر السوق

لوحة التحكم

الطلبات

طلب تسعير جديد

طلبات

المدفوعات

تسهيلات الأسعار

الإمدادات

1# طلب 1#

الإجمالي (شامل الضريبة)

30,707.64 ر.س

مدا

STC Pay

Apple Pay

تحويل بنكي

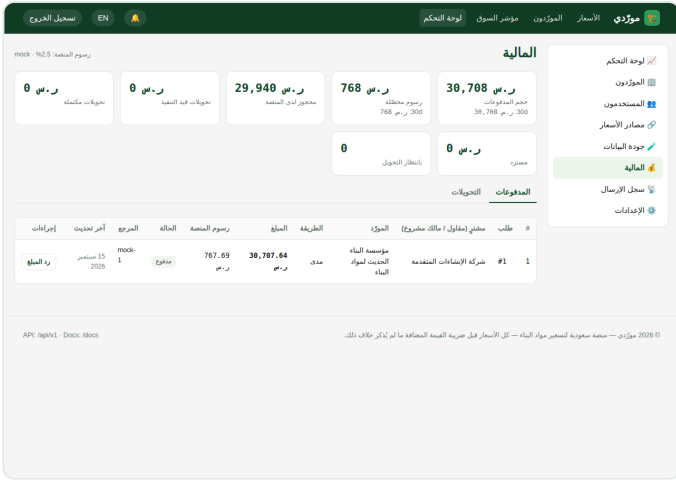
إلغاء التأكيد

إلغاء

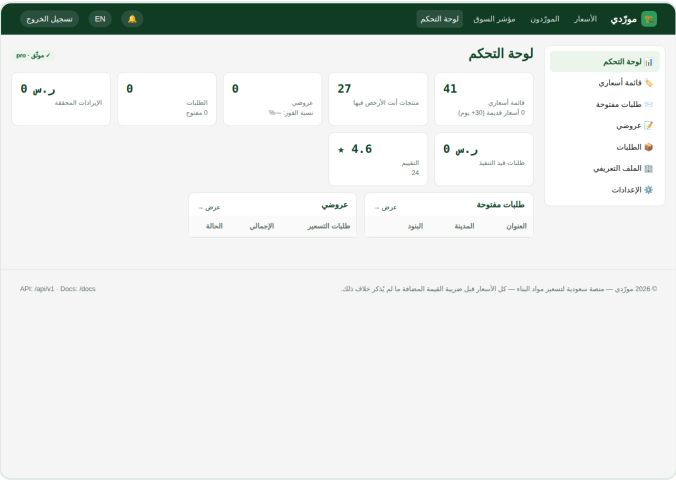
Escrow checkout: mada, card, Apple Pay, STC Pay, bank transfer



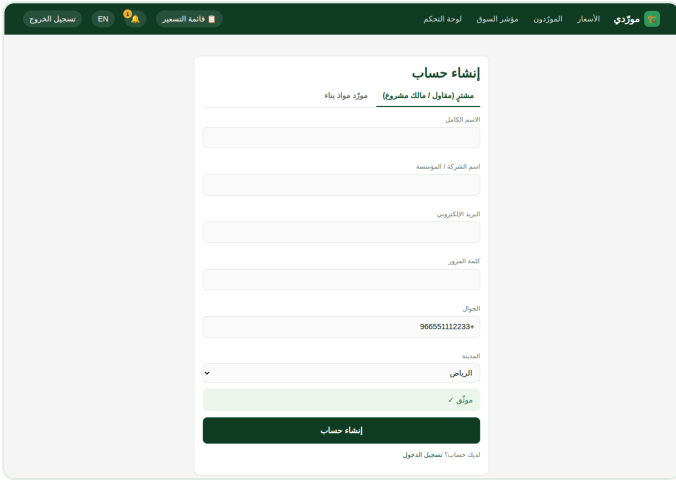
ZATCA phase-1 tax invoice with QR



Admin finance: volume, fees, escrow, payouts



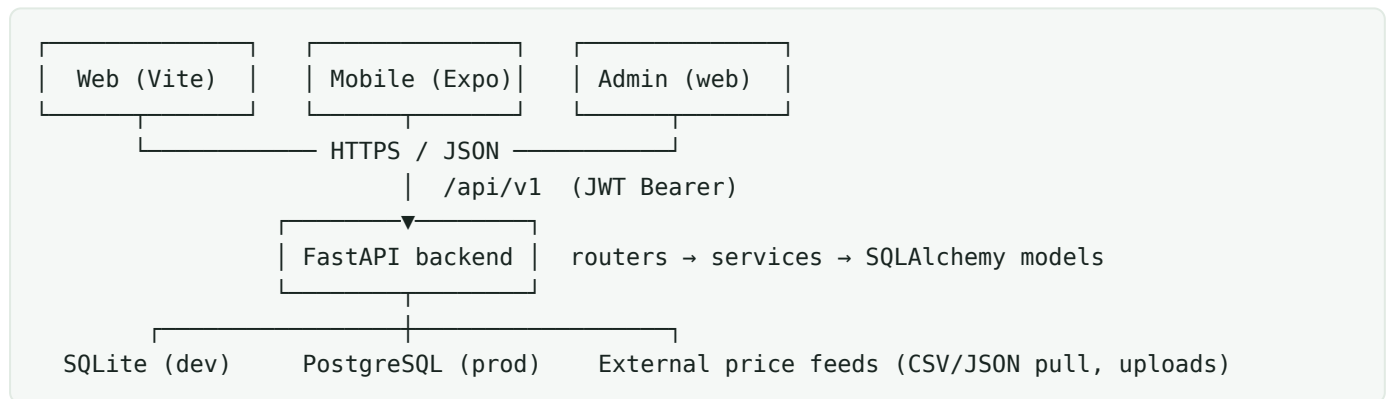
Supplier dashboard



OTP-verified registration

Part 4 — Architecture & data model

Architecture



Domain model

- **User** (buyer | supplier | admin) — a supplier user owns exactly one **Supplier** profile.
- **Supplier** — registered (can bid, receive orders, verified flag, plan) or *external* (`is_external=True` : reference-price source only, created automatically by feed ingestion).
- **Category** (two levels) → **Product** (canonical item: SKU, name ar/en, brand, unit, spec JSON).
- **Offer** — one row per (supplier, product, city): price, VAT flag, min qty, stock, validity, source. Every write appends to **PriceHistory**, which powers the 90-day chart, the 30-day change and the market index.
- **PriceSource** — an external feed definition (URL + kind + city, optional supplier) with fetch status.
- **RFQ** → **RFQItems** (product or free text, qty, unit, target). Visibility public (matched by supplier categories) or invited (**RFQInvite**).
- **Bid** (one per supplier per RFQ; re-submitting replaces) → **BidItems**. Totals = items + delivery fee, + 15% VAT.
- Award → **Order** (+ **OrderItems**) with a fixed state machine `pending` → `confirmed` → `in_delivery` → `delivered` (cancel from pending/confirmed) → **Review** updates supplier rating.
- **Notification** (in-app; `services/notify.register_hook` to add push/email/WhatsApp), **PriceAlert**, **AuditLog**.

Price aggregation (`services/pricing.py`)

- `active_offers_query` : excludes out-of-stock and expired offers; optional city filter.
- `summarize / bulk_summaries` : min / max / avg / median, supplier counts (registered vs external), last update, 30-day change (avg now vs the 7-day window ending 30 days ago).
- `market_index` : per top-level category movement; `trending` : largest absolute 30-day moves.
- Prices are normalised to **ex-VAT** for comparison regardless of how the supplier entered them.

Security

- PBKDF2-SHA256 password hashes (200k iterations), HS256 JWT (7 days), role dependencies (`require_admin`, `require_supplier`, `require_buyer`).
- Suppliers never see competitor bids; buyers only their own RFQs/orders; admin router is fully guarded.
- CORS configurable; secrets via env (`JWT_SECRET` auto-generated into `data/jwt.secret` if unset).

Scaling path

1. Switch `DATABASE_URL` to PostgreSQL (schema is portable; add Alembic migrations before the first production schema change).
2. Run `uvicorn --workers N` behind nginx; move uploads to S3-compatible storage.
3. Add Redis for notification fan-out and a scheduler (cron / Celery beat) that calls `ingestion.fetch_source` for every active `PriceSource` daily.
4. Full-text search (PostgreSQL `tsvector` or Meilisearch) once the catalog exceeds ~50k products.
5. Push notifications: register an Expo push token per device and forward through `notify.register_hook`.

Part 5 — Deployment runbook

Go-live guide — My Supplier · Build for Less

Everything a developer needs to take the platform live. Estimated time for a first production deploy: **about 2 hours** plus waiting on third-party approvals (Moyasar account, Unifonic sender ID, app-store review).

0. What you need

Item	Notes
A Linux server (Ubuntu 22.04+, 2 vCPU / 4 GB is enough to start)	any provider in KSA/region: STC Cloud, Oracle Jeddah, AWS Bahrain, Hetzner...
A domain, e.g. <code>mysupplier.sa</code> , with an A record to the server IP (and <code>www</code>)	HTTPS certificates are automatic (Caddy / Let's Encrypt)
Docker + Docker Compose plugin on the server	<code>curl -fsSL https://get.docker.com \ sh</code>
Moyasar account (live keys)	payments — mada, Visa/MC, Apple Pay, STC Pay
Unifonic account + approved sender ID	SMS OTP and notifications in Saudi Arabia (Twilio also supported)
SMTP account (e.g. Google Workspace, Zoho, SES)	email OTP and notifications
Apple Developer + Google Play accounts, Expo (EAS) account	mobile app publishing

1. Server deploy (Docker, recommended)

```
sudo mkdir -p /opt/mysupplier && sudo chown $USER /opt/mysupplier
git clone https://github.com/invetechsl/My-supplier-.git /opt/mysupplier
cd /opt/mysupplier
cp .env.production.example .env
nano .env          # fill DOMAIN, POSTGRES_PASSWORD, JWT_SECRET (openssl rand -hex 32),
ADMIN_PASSWORD, Moyasar/Unifonic/SMTP keys
docker compose -f docker-compose.prod.yml up -d --build
docker compose -f docker-compose.prod.yml logs -f api      # wait for "Application startup
complete"
```

Open `https://<DOMAIN>` → the site, `https://<DOMAIN>/docs` → API docs. Log in with `ADMIN_EMAIL` / `ADMIN_PASSWORD`, then change the password in Settings.

Updates later: `./deploy/deploy.sh` (pulls main, rebuilds, migrates, restarts). **Backups:** add `0 3 * * *` `/opt/mysupplier/deploy/backup.sh` to cron.

Without Docker: see `deploy/mysupplier.service` (systemd) and put nginx/Caddy in front.

2. Payments (Moyasar)

- In the Moyasar dashboard create **live** API keys → `MOYASAR_SECRET_KEY`, `MOYASAR_PUBLISHABLE_KEY`.
- Enable mada, Visa/Mastercard, Apple Pay (upload the domain-association file if Moyasar asks; served from the site root) and STC Pay.

3. Add a **webhook**: URL `https://<DOMAIN>/api/v1/payments/webhook/moyasar`, events `invoice_paid`, `payment_paid`, `payment_failed`; set a secret token and put it in `MOYASAR_WEBHOOK_SECRET`.
4. Test with one real order of a few riyals: pay → order shows *paid* → tax invoice appears → supplier marks delivered → payout appears in Admin → Finance.
5. Bank transfers need no gateway: admin confirms receipt in Admin → Finance. Set `PLATFORM_BANK_INSTRUCTIONS` to your real IBAN.
6. Payouts to suppliers are done from your bank (manual) and marked paid in the dashboard; automated bank payouts are on the roadmap.

3. OTP & notifications

- **Unifonic**: create an app → `UNIFONIC_APP_SID`; register the sender ID (`UNIFONIC_SENDER_ID`) with CST through Unifonic. Set `SMS_PROVIDER=unifonic`.
- **Email**: `EMAIL_PROVIDER=smtp` + SMTP settings. Add SPF/DKIM records for the sending domain.
- **WhatsApp** (optional): `WHATSAPP_PROVIDER=meta` with a WhatsApp Cloud API number, or `twilio`.
- **Push**: works automatically for EAS builds (`PUSH_PROVIDER=expo`).
- Set `OTP_REQUIRED=1` and `OTP_DEBUG_RETURN_CODE=0` in production (codes are never returned by the API).
- Admin → *Delivery log* shows every SMS/email/push with provider references and lets you retry failures.

4. Mobile apps (iOS + Android)

```
cd mobile
npm install --legacy-peer-deps
npm i -g eas-cli && eas login
eas init # writes the EAS projectId into app.json → extra.eas.projectId
# point the app at production
# app.json → expo.extra.apiBase = "https://<DOMAIN>" (or EXPO_PUBLIC_API_BASE in eas.json
env)
eas build -p android --profile production # .aab for Google Play
eas build -p ios --profile production # needs Apple Developer account
eas submit -p android / eas submit -p ios
```

Store listing: name **موّردِي — Build for Less**, icon and splash are in `mobile/assets/`, bundle IDs `sa.mysupplier.app` (change in `app.json` if needed). Push notifications require the EAS build (Expo Go is fine for testing everything else).

5. Go-live checklist

- [] `.env`: `SEED_DEMO_DATA=0`, strong `JWT_SECRET`, `ADMIN_PASSWORD` changed after first login, `CORS_ORIGINS` = your domain
- [] HTTPS working (`https://<DOMAIN>/api/v1/health` returns `{"status":"ok"}`)
- [] Moyasar live keys + webhook secret set; one real test payment done and refunded
- [] Unifonic sender ID approved; OTP received on a real phone; `OTP_REQUIRED=1`
- [] SMTP verified (registration email arrives, not in spam; SPF/DKIM set)
- [] `PLATFORM_VAT_NUMBER` and legal name set (they print on invoices/QR)
- [] Categories reviewed in Admin; first real suppliers invited and verified; demo suppliers not present
- [] External price sources added in Admin → Price sources (CSV/JSON links or uploads)
- [] Nightly backup cron installed; a restore tested once (`gunzip -c file.sql.gz | docker compose -f docker-compose.prod.yml exec -T db psql -U mysupplier mysupplier`)

- [] Mobile builds submitted; `apiBase` points to production
- [] Monitoring: uptime check on `/api/v1/health` ; optionally forward container logs to your log service

6. Operating the platform

- **Admin dashboard** (`/admin`): KPIs, supplier verification and plans, users, price sources, data quality, finance (payments, transfers, refunds, payouts), delivery log and scheduled jobs.
- **Scaling**: raise `WEB_CONCURRENCY` ; move PostgreSQL to a managed instance (set `DATABASE_URL`); with several API replicas keep `JOBS_ENABLED=1` on exactly one of them (or run `python -m app.jobs` from cron) and move rate limiting to Redis (roadmap).
- **Migrations**: after any model change run `alembic revision --autogenerate -m "..."` locally, commit, and `deploy.sh` applies it.
- **Docs**: `docs/ARCHITECTURE.md` , `docs/API.md` , `docs/INTEGRATIONS.md` , `docs/BUSINESS_MODEL.md` , `docs/ROADMAP.md` , `brand/README.md` .

Part 6 — Integrations & environment variables

Integrations: payments, OTP, notifications, jobs

Everything below works out of the box in **dev mode** with built-in stand-ins (mock gateway, console SMS/email, console push) and switches to real providers by environment variables only. No code changes are needed to go live.

1. Payments (escrow) — `backend/app/services/payments.py`

Flow: buyer clicks *Pay now* on an order → `POST /payments/checkout {order_id, method}` → the platform creates a `Payment` and returns a hosted checkout URL (or bank-transfer instructions) → gateway confirms via **webhook** and/or the client calls `POST /payments/{id}/sync` after redirect → order becomes `paid`; a ZATCA-style **tax invoice** (supplier → buyer) and a **platform-fee invoice** (platform → supplier) are issued → supplier delivers → delivered **releases escrow** into a `Payout` (net of the platform fee) → admin pays the supplier and marks the payout paid (reference stored). Cancelling a paid, undelivered order refunds automatically.

Variable	Meaning
<code>PAYMENT_PROVIDER</code>	<code>mock</code> (default, dev) or <code>moyasar</code>
<code>MOYASAR_SECRET_KEY</code> / <code>MOYASAR_PUBLISHABLE_KEY</code>	from the Moyasar dashboard (mada, Visa/MC, Apple Pay, STC Pay)
<code>MOYASAR_WEBHOOK_SECRET</code>	secret you set on the Moyasar webhook; sent as <code>X-Webhook-Secret</code> header or <code>secret_token</code> field. Webhook URL: <code>https://<api>/api/v1/payments/webhook/moyasar</code>
<code>PLATFORM_FEE_PCT</code>	take rate (default 2.5) deducted from the supplier payout
<code>PLATFORM_VAT_NUMBER</code>	appears on platform-fee invoices and the QR
<code>PLATFORM_BANK_INSTRUCTIONS</code>	shown for <code>bank_transfer</code> ; admin confirms receipt in Admin → Finance
<code>PUBLIC_BASE_URL</code> / <code>WEB_BASE_URL</code>	used for callback and return URLs

The webhook never trusts the payload alone: with `moyasar` the payment status is re-fetched from the Moyasar API before it is marked paid. Adding HyperPay/Tap/Tabby = one more provider class with `create_checkout` / `fetch_status` / `refund`.

Invoices: `GET /payments/invoices/order/{id}` (JSON) and `GET /payments/invoices/{id}.html` (printable, QR as SVG). The QR is the ZATCA phase-1 TLV (seller, VAT no., timestamp, total, VAT). Phase-2 (XML + signing + clearance) is a later step and slots into `issue_invoices`.

2. OTP — `backend/app/services/otp.py`

`POST /auth/otp/request {destination, purpose, channel?}` → `POST /auth/otp/verify {destination, code, purpose}` → `verification_token` (15 min).

- `purpose=register`: pass the token as `otp_token` to `/auth/register` → phone/email is stored as verified. Set `OTP_REQUIRED=1` to make it mandatory.
- `purpose=login`: verify returns an access token directly (passwordless login by phone).
- `purpose=reset`: use the token with `POST /auth/password/reset`.

- `purpose=verify` : logged-in user attaches a verified phone via `POST /auth/verify-contact` .

Codes are 6 digits, hashed (HMAC), expire after `OTP_TTL_MINUTES` (5), max `OTP_MAX_ATTEMPTS` (5), at most `OTP_MAX_REQUESTS_PER_10MIN` (3) per destination, plus a per-IP limit (`RATE_LIMIT_OTP_PER_MINUTE`). Saudi numbers are normalised to `+9665XXXXXXX` . While the SMS/email provider is `console` , the API returns `debug_code` so the UI flow works without a real provider (`OTP_DEBUG_RETURN_CODE=0` disables this).

3. Notification channels — `backend/app/services/channels.py`

Every in-app notification also creates one **outbox row per enabled channel** (user preferences: `PATCH /auth/me/notifications`). A background worker sends them with retries (30s, 60s, 120s) and records provider references and errors. Admin → *Delivery log* shows every send and can retry failures.

Channel	*_PROVIDER	Variables
Email	<code>console</code> / <code>smtp</code>	<code>SMTP_HOST</code> , <code>SMTP_PORT</code> , <code>SMTP_USER</code> , <code>SMTP_PASSWORD</code> , <code>SMTP_FROM</code> , <code>SMTP_TLS</code>
SMS	<code>console</code> / <code>unifonic</code> / <code>twilio</code>	<code>UNIFONIC_APP_SID</code> , <code>UNIFONIC_SENDER_ID</code> or <code>TWILIO_ACCOUNT_SID</code> , <code>TWILIO_AUTH_TOKEN</code> , <code>TWILIO_FROM</code>
WhatsApp	<code>console</code> / <code>twilio</code> / <code>meta</code>	<code>TWILIO_WHATSAPP_FROM</code> or <code>META_WA_PHONE_ID</code> , <code>META_WA_TOKEN</code> (WhatsApp Cloud API)
Push	<code>expo</code> / <code>console</code>	optional <code>EXPO_ACCESS_TOKEN</code> . Devices register via <code>POST /notifications/devices</code> (the mobile app does this after login)

Unifonic is the usual choice in Saudi Arabia (sender-ID registration required with CST); Twilio works for both SMS and WhatsApp during pilots. WhatsApp outbound outside the 24-hour window needs approved templates — swap the `text` body for a `template` payload in `MetaWhatsApp.send` .

4. Background jobs — `backend/app/services/jobs.py`

Runs inside the API process (`JOBS_ENABLED=1` , tick every `JOBS_TICK_SECONDS`) or from cron: `python -m app.jobs` (all) / `python -m app.jobs fetch_feeds` .

Job	Interval	What
<code>outbox</code>	every tick	send queued notification deliveries
<code>close_expired_rfqs</code>	5 min	close RFQs past <code>closes_at</code> , tell the buyer how many bids arrived
<code>price_alerts</code>	15 min	notify watchers when the best price hits their target (or drops 3% below baseline)
<code>fetch_feeds</code>	hourly check, <code>FEED_FETCH_INTERVAL_HOURS</code>	pull every active external price source
<code>expire_payments</code>	hourly	fail checkout attempts older than 24h

Admin → *Delivery log* lists recent runs and can run any job now (`POST /admin/jobs/{name}/run`). With several API replicas, run jobs from one cron host and set `JOBS_ENABLED=0` on the others.

5. Rate limiting & migrations

- Per-IP limits on `POST /api/v1/auth/*` (`RATE_LIMIT_AUTH_PER_MINUTE` , 30) and `/auth/otp/*` (`RATE_LIMIT_OTP_PER_MINUTE` , 6). In-memory; behind a load balancer use `X-Forwarded-For` (already honoured) or move the counter to Redis.
- **Alembic** is set up in `backend/migrations` (initial revision included). Production: `AUTO_CREATE_TABLES=0` and run `alembic upgrade head` on deploy; after model changes `alembic revision --autogenerate -m "..."`.

6. Mobile specifics

- Expo push: `expo-notifications` + `expo-device` ; token is registered after login (`src/push.ts`). Build with EAS so the Expo push token maps to APNs/FCM (`eas credentials`).
- Payments open the hosted checkout in the system browser; after payment the buyer returns to the app and the order list refreshes (`/payments/{id}/sync` is called by the web return page; the app re-reads the order).

7. File storage (v1.2)

Uploads (supplier logos, product images, compliance documents) go to local disk by default (`backend/data/uploads` , served at `/uploads/...`). For production set `STORAGE_BACKEND=s3` with `S3_BUCKET` , `S3_REGION` , optional `S3_ENDPOINT` (Oracle OCI, Wasabi, MinIO) and `S3_PUBLIC_BASE` (CDN), plus the standard `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` . Files are type-checked by magic bytes (PNG/JPG/WEBP, PDF for documents) and limited to `MAX_UPLOAD_MB` (8). Documents are stored with random names; for stricter privacy move them to a private bucket and serve signed URLs (see the handover backlog).

8. Monitoring

Set `SENTRY_DSN` (and `ENVIRONMENT=production`) to send exceptions and traces to Sentry; nothing is sent when unset.

Complete environment variable reference (backend/.env.example)

```
# Database – SQLite by default (backend/data/mysupplier.db). For production use Postgres:
# DATABASE_URL=postgresql+psycopg://user:password@host:5432/mysupplier
DATABASE_URL=

# Long random string. Auto-generated into backend/data/jwt.secret when empty.
JWT_SECRET=

# First admin account (created on first start)
ADMIN_EMAIL=admin@mysupplier.sa
ADMIN_PASSWORD=Admin@2026

# 1 = create demo suppliers/prices on first start, 0 = catalog + admin only
SEED_DEMO_DATA=1

# Comma separated allowed web origins, * for all
CORS_ORIGINS=*
VAT_RATE=0.15

# ---- Payments (see docs/INTEGRATIONS.md) ----
PAYMENT_PROVIDER=mock           # mock | moyasar
MOYASAR_SECRET_KEY=
MOYASAR_PUBLISHABLE_KEY=
MOYASAR_WEBHOOK_SECRET=
PLATFORM_FEE_PCT=2.5
PLATFORM_VAT_NUMBER=3000000000000003
PLATFORM_BANK_INSTRUCTIONS=Bank: Al Rajhi Bank – IBAN: SA00 0000 0000 0000 0000 0000 –
Beneficiary: My Supplier Co.
PUBLIC_BASE_URL=http://localhost:8000
WEB_BASE_URL=http://localhost:8000

# ---- OTP ----
OTP_REQUIRED=0                  # 1 = phone/email verification mandatory at registration
OTP_DEBUG_RETURN_CODE=auto      # auto: return the code in the API response only while the
provider is "console"

# ---- Notification channels ----
EMAIL_PROVIDER=console          # console | smtp
SMTP_HOST=
SMTP_PORT=587
SMTP_USER=
SMTP_PASSWORD=
SMTP_FROM=no-reply@mysupplier.sa
SMS_PROVIDER=console            # console | unifonic | twilio
UNIFONIC_APP_SID=
UNIFONIC_SENDER_ID=MySupplier
TWILIO_ACCOUNT_SID=
TWILIO_AUTH_TOKEN=
TWILIO_FROM=
WHATSAPP_PROVIDER=console       # console | twilio | meta
TWILIO_WHATSAPP_FROM=
META_WA_PHONE_ID=
META_WA_TOKEN=
PUSH_PROVIDER=expo              # expo | console
EXPO_ACCESS_TOKEN=

# ---- Jobs / limits / migrations ----
JOBS_ENABLED=1
```

```
FEED_FETCH_INTERVAL_HOURS=24
RATE_LIMIT_AUTH_PER_MINUTE=30
RATE_LIMIT_OTP_PER_MINUTE=6
AUTO_CREATE_TABLES=1          # 0 in production; run: alembic upgrade head

# ---- File storage & monitoring ----
STORAGE_BACKEND=local          # local | s3
S3_BUCKET=
S3_REGION=me-south-1
S3_ENDPOINT=
S3_PUBLIC_BASE=
MAX_UPLOAD_MB=8
SENTRY_DSN=
ENVIRONMENT=development
```

Part 7 — API reference

API reference (v1)

Base URL: `/api/v1`. Auth: Authorization: Bearer `<token>` from `/auth/login` or `/auth/register`.
Interactive docs at `/docs`.

Auth

Method	Path	Who	Notes
POST	<code>/auth/register</code>	public	<code>{email,password,full_name,role:buyer supplier,company_name,city,phone,cr_number,category_ids}</code> → token + user
POST	<code>/auth/login</code>	public	<code>{email,password}</code>
GET / PATCH	<code>/auth/me</code>	any	profile
POST	<code>/auth/change-password</code>	any	

Catalog & market (public)

Method	Path	Notes
GET	<code>/catalog/categories</code>	tree with product counts
GET	<code>/catalog/products</code>	<code>q, category_id, city, brand, sort=relevance price_asc price_desc offers, page, size, only_with_offers</code> → items with summary
GET	<code>/catalog/products/{id}</code>	offers sorted by ex-VAT price, 90-day history, related
GET	<code>/catalog/compare?ids=1,2,3</code>	side-by-side
GET	<code>/catalog/brands</code> · <code>/catalog/cities</code>	filter values
POST/PUT	<code>/catalog/products</code>	supplier/admin create or edit canonical products
GET/POST/DELETE	<code>/catalog/alerts</code>	price watchlist (auth)
GET	<code>/market/stats</code> · <code>/market/index</code> · <code>/market/trending</code>	landing-page stats, category index, biggest moves

Suppliers

Method	Path	Who
GET	<code>/suppliers</code> · <code>/suppliers/{id}</code> · <code>/suppliers/{id}/offers</code>	public

Method	Path	Who
GET/PUT	/suppliers/me	supplier
GET	/suppliers/me/dashboard	supplier KPIs
GET/POST	/suppliers/me/offers	list / upsert (product_id + city unique)
PATCH/DELETE	/suppliers/me/offers/{id}	
POST	/suppliers/me/offers/import	multipart CSV/JSON — columns: sku, product_name, name_ar, category, unit, price, city, min_qty, includes_vat, stock_status, notes

RFQ & bids

Method	Path	Who
POST	/rfq	buyer — {title, city, delivery_address, needed_by, closes_at, visibility, invited_supplier_ids, items: [{product_id description, quantity, unit, target_price}], publish}
GET	/rfq/mine	buyer
GET	/rfq/open	supplier — public RFQs matching its categories + invitations (only_matching=false for all)
GET	/rfq/{id}	owner sees ranked bids; supplier sees items + my_bid only
POST	/rfq/{id}/publish · /close	buyer
POST	/rfq/{id}/bids	supplier — {items:[{rfq_item_id, unit_price, quantity?, brand}], delivery_days, delivery_fee, valid_until, payment_terms, notes}; resubmitting replaces
POST	/rfq/{id}/bids/{bid_id}/withdraw	supplier
POST	/rfq/{id}/award/{bid_id}	buyer → creates order, rejects other bids, notifies
GET	/rfq/bids/mine	supplier

Orders

Method	Path	Who
POST	/orders/direct	buyer — {offer_id, quantity, delivery_address} (registered suppliers only)
GET	/orders/mine · /orders/supplier · /orders/{id}	
PATCH	/orders/{id}/status	supplier: confirmed → in_delivery → delivered / cancelled; buyer: cancelled (pending only)
POST	/orders/{id}/review	buyer, delivered orders — {rating 1-5, comment}

Notifications

GET /notifications?unread_only= , GET /notifications/unread-count , POST /notifications/read-all , POST /notifications/{id}/read .

Admin (role=admin)

GET /admin/dashboard · GET/PATCH /admin/users · GET /admin/suppliers?verified= · POST /admin/suppliers/{id}/verify?verified=&plan= · POST/PUT /admin/categories · DELETE /admin/products/{id} (deactivate) · GET /admin/price-alerts (stale + outliers) · GET/POST/DELETE /admin/sources · POST /admin/sources/{id}/fetch · POST /admin/sources/{id}/upload · GET /admin/audit .

Payments (v1.1)

Method	Path	Who	Notes
POST	/payments/checkout	buyer	{order_id, method: mada card applepay stcpay bank_transfer} → payment with checkout_url or bank_instructions
GET	/payments/mine · /payments/order/{id}	buyer / parties	
POST	/payments/{id}/sync · /payments/{id}/cancel	buyer	reconcile with gateway / cancel open attempt
POST	/payments/webhook/moyasar	gateway	X-Webhook-Secret header or secret_token body
GET	/payments/invoices/order/{id} · /payments/invoices/{id}.html	parties	JSON / printable with QR
GET	/payments/payouts/mine · /payments/supplier/summary	supplier	
GET	/admin/finance · /admin/payments · /admin/payouts	admin	
POST	/admin/payments/{id}/confirm-transfer · /admin/payments/{id}/refund · /admin/payouts/{id}/paid	admin	

OTP & account (v1.1)

POST /auth/otp/request {destination, purpose, channel?} · POST /auth/otp/verify {destination, code, purpose} · POST /auth/password/reset {verification_token, new_password} · POST /auth/verify-contact?verification_token= · PATCH /auth/me/notifications {notify_email, notify_sms, notify_whatsapp, notify_push} . /auth/register accepts otp_token .

Notifications & ops (v1.1)

POST /notifications/devices {token, platform, device_name} · DELETE /notifications/devices/{token} · GET /notifications/deliveries · admin: GET /admin/deliveries?status= · POST /admin/deliveries/{id}/retry · GET /admin/jobs · POST /admin/jobs/{name}/run .

v1.2: uploads, BOQ, disputes

POST /suppliers/me/logo (multipart) · GET/POST /suppliers/me/documents?
kind=cr|vat|classification|bank|other · POST /catalog/products/{id}/image · POST /rfq/import-boq
(xlsx/csv → items with suggested product_id) · GET /rfq/{id}/export.xlsx · POST /orders/{id}/dispute
{reason} · GET /orders/{id}/disputes · admin: GET /admin/documents?status= · POST
/admin/documents/{id}/review {status, note, expires_at} · GET /admin/disputes?status= · POST
/admin/disputes/{id}/resolve {status, resolution, refund}.

Interactive documentation with request/response schemas is served by the API at /docs (Swagger) and /redoc .

Part 8 — Remaining backlog after launch

Prioritised work after the website is live. Each item names the files to touch and what "done" means.

P0 — first weeks after launch

Item	Details	Est.
Mobile app on real devices	Run on iOS/Android via Expo Go and an EAS development build; force RTL (<code>I18nManager.forceRTL + reload</code> ; only <code>allowRTL</code> today); refresh orders on app focus after returning from the hosted checkout (<code>/payments/{id}/sync</code>); add parity for BOQ import, document upload and disputes; EAS production builds; store listings with <code>mobile/assets</code> . Done when TestFlight and Play internal-testing builds pass a full buyer and supplier flow.	3–5 d
Private document storage	<code>services/storage.py</code> : <code>save_private + signed_url</code> (boto3 <code>generate_presigned_url</code>); authenticated document endpoint in <code>routers/suppliers.py</code> / <code>routers/admin.py</code> ; web links use it.	1 d
Multi-replica safety	Redis counters for rate limiting (<code>main.py rate_limit</code>); run jobs from one cron host (<code>python -m app.jobs</code> , <code>JOBS_ENABLED=0</code> on replicas) or a Redis lock in <code>services/jobs.py tick()</code> .	1 d
Automated UI tests	Playwright suite replaying the UAT script against a seeded API in CI (<code>.github/workflows/ci.yml</code>).	2 d
Refresh tokens	Access tokens live 7 days; add short-lived access + refresh token rotation in <code>security.py</code> and <code>routers/auth.py</code> , update <code>web/src/api.ts</code> and <code>mobile/src/api.ts</code> .	1 d

P1 — first 3 months

Item	Details	Est.
ZATCA phase-2 e-invoicing	UBL XML, CSID signing, clearance/reporting API; extend <code>services/payments.issue_invoices</code> , store cleared XML/hash on <code>Invoice</code> ; consider a certified provider SDK.	2–3 w
BNPL & second gateway	Tabby/Tamara and HyperPay: one provider class each in <code>services/payments.py</code> (<code>create_checkout</code> / <code>fetch_status</code> / <code>refund</code>) + webhook route + method option in <code>schemas.CheckoutIn</code> .	1 w each
Automated payouts	Bank API transfers batched by a job; replaces manual "mark paid" in <code>routers/admin.py</code> .	2 w
Search & catalog quality	PostgreSQL full-text search (<code>services/pricing.search_products</code> uses <code>ILIKE</code>), product merge/de-dup tools, unit normalisation table, Arabic synonyms; buyer VAT number field.	2 w
Supplier connectors	Guided Google Sheets link, ERP adapters (Odoo, SAP B1) as new <code>PriceSource.kind</code> values in <code>services/ingestion.py</code> , per-supplier CSV templates.	1–2 w
Buyer teams & projects	Company model with members, project folders for RFQs/orders, PDF export of comparisons.	2 w
Trust & growth	Document expiry reminders job, verified-documents badge, review moderation, WhatsApp templates for Meta, SEO pre-rendering of product/city pages, referral credits, featured placement by plan.	2–3 w

P2 — later

- Price forecasts and buy/wait signals, city arbitrage, public weekly index.
- Logistics marketplace, financing partners, GCC expansion (multi-currency/country).
- Native mobile extras (camera BOQ scan, offline cache), tablet layouts.

Known limitations

- Money stored as rounded floats; move to `Numeric(14,2)` when adding accounting exports.
- Escrow is logical (funds sit in your Moyasar/bank account); reconcile with Admin → Finance and keep a separate settlement account.
- Invoices are ZATCA phase 1 (QR). External price feeds expect the documented CSV/JSON columns; no website scraping by design.
- Notification worker runs inside the API process; queued rows survive restarts. In-memory rate limiting is per container.
- The repository branch `claude/building-materials-ecommerce-ktfxj3` is an unrelated experiment with failing CI — deploy from `main` and delete it.

Part 9 — Business model & roadmap

Business model & fundraising path

The market

Saudi construction spending is in the hundreds of billions of riyals per year (giga-projects, housing, infrastructure). Building materials are typically 50–60% of project cost, yet pricing is opaque: contractors phone 5–10 suppliers per item, prices differ 10–25% between suppliers and cities, and quotes take days. A single trusted, always-current price layer plus an RFQ marketplace is the wedge.

Revenue lines (all implemented or ready to switch on)

Line	Mechanism	Where in the product
Transaction take-rate	1.5–3% of awarded RFQ / direct-order GMV, charged to the supplier	Orders carry totals; admin dashboard shows GMV and estimated revenue at 2.5%
Supplier subscriptions	free / pro / enterprise plans (bid limits, featured placement, analytics, multi-city price lists, API access)	<code>Supplier.plan</code> , admin sets plan on verification
Verification & compliance	paid onboarding/verification badge (CR, VAT, ratings)	<code>Supplier.verified</code>
Price intelligence	paid market-index reports, category trend data, API access for developers, ERP integrations	<code>/market/index</code> , <code>/market/trending</code> , price history tables
Buyer premium	project-level RFQ workspaces, team seats, invited-only tenders, procurement analytics	RFQ visibility + invites; extend with teams
Financing & logistics (later)	BNPL for materials with partner banks, delivery marketplace, escrow	Order state machine is the hook

Why buyers and suppliers come (and stay)

- Buyer: one search shows *all* prices; one RFQ reaches every relevant supplier; awarded orders are tracked and rated.
- Supplier: qualified demand delivered to them, zero cost to list, visibility of where they rank on price (`best_price_offers` on their dashboard), stale-price nudges keep the data fresh.
- Data flywheel: more suppliers → better comparisons → more buyers → more RFQs → more suppliers. External price feeds seed the comparison before the supplier base is dense.

KPIs to show investors (all available in the admin dashboard / DB)

- Priced products and offers, share of offers updated in the last 30 days (freshness).
- Registered vs verified suppliers, RFQ count, bids per RFQ, award rate, time-to-first-bid.
- GMV (awarded + direct), take-rate revenue, repeat-buyer rate, supplier win-rate distribution.

Path to a large outcome

The ask of "billions of riyals" is a **GMV** target, not first-year revenue. A realistic ladder: 1. **0–6 months (Riyadh, 3 categories: cement, steel, blocks/aggregates)**: 200+ verified suppliers, 3,000+ priced SKUs, 100 RFQs/month.

Raise pre-seed/seed on freshness + RFQ traction. 2. **6–18 months (5 cities, all categories, mobile in stores, API):** 1,000+ suppliers, take-rate switched on, first ERP/e-procurement integrations with large contractors, price-index licensing. Series A on GMV run-rate and supplier retention. 3. **18–36 months:** financing partner, logistics marketplace, GCC expansion. At 2.5% take-rate, SAR 1B annual GMV ≈ SAR 25M revenue plus subscriptions and data — this is the scale at which "billions in GMV" becomes a credible, financeable story.

Fundraising conversations in KSA should reference Vision 2030 housing targets, the Contractors Authority (SCA) classification data for supplier onboarding, and Etimad/Monsha'at programs for SME suppliers.

Roadmap

Shipped in v1.0

- Backend API (FastAPI): auth/roles, catalog with multi-supplier price comparison, price history & market index, external price feed ingestion (CSV/JSON URL + upload), supplier price lists (+ bulk import), RFQ with public/invited visibility, bidding with market reference, award → order → delivery → review, direct orders, price alerts, notifications, admin KPIs, verification, data-quality alerts, audit log. 8 end-to-end tests.
- Web (React): public site (home, catalog, product comparison, suppliers, market index), buyer workspace, supplier portal, admin dashboard. Arabic RTL / English.
- Mobile (Expo iOS/Android): search & compare, product detail with direct order, quote list → RFQ, RFQ list/detail with bidding (supplier) and awarding (buyer), orders, notifications, supplier price list, auth.
- Docker Compose (PostgreSQL + API serving the built web), GitHub Actions CI.

Shipped in v1.1

- Escrow payments (Moyasar hosted checkout: mada, cards, Apple Pay, STC Pay; bank transfer; mock gateway for dev), platform fee, payouts, refunds, ZATCA phase-1 tax invoices with QR, admin finance dashboard, supplier payout view.
- OTP by SMS/WhatsApp/email (Unifonic, Twilio, SMTP): verified registration, passwordless phone login, password reset, phone verification.
- Multi-channel notifications (email, SMS, WhatsApp, Expo push) with per-user preferences, outbox with retries and admin delivery log; mobile push registration.
- Background jobs (RFQ expiry, price alerts, feed fetching, payment expiry), per-IP rate limiting, Alembic migrations.

Shipped in v1.2

- File uploads (local disk or S3-compatible storage): supplier logos, product images, supplier compliance documents (CR, VAT, classification, IBAN letter) with admin review and expiry.
- Bill-of-quantities import (Excel/CSV) into an RFQ with automatic product matching; bid comparison export to Excel.
- Order disputes (buyer or supplier) with admin resolution and optional refund; Sentry error monitoring hook.

Next (in priority order)

1. **Production hardening:** Redis-backed rate limiting and job locks for multi-replica deployments; private bucket + signed URLs for supplier documents.
2. **Payments phase 2:** ZATCA phase-2 e-invoicing (XML, signing, clearance), Tabby/Tamara BNPL for buyers, automated payouts via bank API, HyperPay as a second gateway.

3. **Supplier connectors:** per-supplier CSV templates and adapters for common supplier ERPs (Odoo, SAP B1) and Google Sheets links (a published-CSV Sheets link already works as a price source).
4. **Search & scale:** PostgreSQL full-text search, product de-duplication (same item, different brands/names), unit normalisation (ton ↔ bag ↔ m³).
5. **Buyer teams & projects:** multiple users per company, per-project RFQ folders, PDF export of comparisons.
6. **Trust:** document expiry reminders, supplier badges from approved documents, review moderation.
7. **Intelligence:** price forecasts per category, "should you buy now" signals, city arbitrage view, public weekly index page for SEO.
8. **Growth:** referral credits, WhatsApp bot for quick price lookups, Arabic SEO landing pages per product/city.

Part 10 — Brand

Brand — My Supplier · Build for Less

File	Use
logo.svg / logo.png	full lockup: mark + موردي + MY SUPPLIER + Build for Less (web header, docs, social)
mark.svg	square mark only: roof over bricks with a down-arrow (= lower cost). Favicon, app icon
../mobile/assets/icon.png , adaptive-icon.png , splash.png	generated from the mark for iOS/Android

Slogan: Build for Less (always in English, title case) · Arabic line: **أبن بأقل تكلفة** . **Colors:** deep green #0F3D24 , primary #175934 , accent #2E9E5B , light #C6E5D1 . **Fonts:** Noto Kufi Arabic (headings/Arabic), IBM Plex Sans Arabic (body, slogan).

Where the slogan appears: web header and footer, hero, login/register, browser title and Open Graph tags, mobile splash and headers, OTP messages, email footer, invoices, API title and /health , /market/stats .



Logo lockup — Build for Less

Appendix — Demo accounts, commands, file layout

Role	Email	Password
Admin	admin@mysupplier.sa	Admin@2026
Buyer	buyer@demo.sa	Demo@2026
Suppliers	supplier1@demo.sa ... supplier7@demo.sa	Demo@2026

Demo accounts exist only when `SEED_DEMO_DATA=1` (development). Production must run with `SEED_DEMO_DATA=0`.

```
# local development
cd backend && python3 -m venv .venv && source .venv/bin/activate && pip install -r
requirements.txt && uvicorn app.main:app --reload
cd web && npm install && npm run dev
cd mobile && npm install --legacy-peer-deps && npx expo start
# tests / checks
cd backend && python -m pytest -q          # 17 scenarios
cd web && npm run typecheck && npm run build
cd mobile && npm run typecheck
# production
cp .env.production.example .env && docker compose -f docker-compose.prod.yml up -d --build
./deploy/deploy.sh          # update      ·      ./deploy/backup.sh    # nightly backup
# database migrations after model changes
cd backend && alembic revision --autogenerate -m "describe change" && alembic upgrade head
```

```
backend/app/{main.py, config.py, models.py, schemas.py, security.py, seed.py, db.py}
backend/app/routers/{auth, catalog, market, suppliers, rfq, orders, payments, notifications,
admin}.py
backend/app/services/{pricing, ingestion, matching, notify, channels, otp, payments, jobs,
storage, boq}.py
backend/tests/ backend/migrations/ backend/Dockerfile backend/entrypoint.sh
web/src/{App.tsx, api.ts, i18n.tsx, auth.tsx, styles.css} web/src/components/
web/src/pages/{buyer, supplier, admin}/
mobile/App.tsx mobile/src/{api.ts, store.tsx, i18n.ts, theme.ts, push.ts} mobile/src/screens/
mobile/assets/
deploy/{Caddyfile, deploy.sh, backup.sh, mysupplier.service} docker-compose.yml docker-
compose.prod.yml
docs/{ARCHITECTURE, API, INTEGRATIONS, BUSINESS_MODEL, ROADMAP}.md DEPLOYMENT.md README.md
brand/
```

Build for Less.